STT3795 – HIVER 2026

DÉTECTION D'ANOMALIES INDUSTRIELLES : IMPACT DU DÉBRUITAGE SPATIAL (DBSCAN) SUR L'APPRENTISSAGE PAR SVM

PAR
YVES SERY

TRAVAIL PRÉSENTÉ À GUY WOLF
DANS LE CADRE DU COURS STT 3795
FONDEMENTS THÉORIQUES EN SCIENCES DES DONNÉES

AVRIL 2026

Université de Montréal

Table des matières

TABLE DES FIGURES 1
TABLEAUX 1
INTRODUCTION 2
I. OBJECTIFS 2
II. DESCRIPTION DES DONNÉES ANALYTIQUES 3
A. SOURCE ET STRUCTURE DE DONNÉES 3
Caractéristiques des données *3*
B. EXPLORATION VISUELLE (PCA) ET JUSTIFICATION DU DÉBRUITAGE 3
III. MÉTHODOLOGIE 5
A. PRÉTRAITEMENT 5
B. DIVISION DES DONNÉES (SPLIT TRAIN/TEST) 5
C. PARAMÈTRES DE DBSCAN 6
Détermination de minPts *6*
Détermination de ε *6*
D. DBSCAN 7
E. SVM 8
Optimisation des hyperparamètres *8*
IV. RÉSULTATS 9
A. RÉSULTATS OBTENUS APRÈS DBSCAN 9
B. COMPARAISON DES DEUX MODÈLES SVM 10
C. INTERPRÉTATION SUR L'HYPOTHÈSE DE RECHERCHE 11
V. CONCLUSION 11
VI. CONTRIBUTIONS 11
BIBLIOGRAPHIE 12

Table des figures

1 - PCA - VARIANCE EXPLIQUÉE CUMULÉE 4
2 - PROJECTION PCA 2D 4
FIGURE 3 - PARAMÈTRES DE DBSCAN 6
FIGURE 4 - GRAPHIQUE K-DISTANCE - CHOIX D'EPSILON 6
FIGURE 5 - ALGORITHME DBSCAN 7
FIGURE 6 - PARAMÈTRES SVM 8
FIGURE 7 - RÉSULTATS DBSCAN 9
FIGURE 8 - MATRICES DE CONFUSION 10

Tableaux

TABLEAU 1 - CARACTÉRISTIQUES DES DONNÉES 3
TABLEAU 2 - F1-SCORE ET AUC-ROC 10

Introduction

Dans le domaine manufacturier de haute précision (comme l'industrie aérospatiale), la qualité des pièces usinées est soumise à des tolérances très strictes. Une pièce « non conforme » entraîne des retards critiques dans la chaîne d'approvisionnement et peut avoir des conséquences graves sur la sécurité globale des systèmes.
L'anticipation de ces défaillances par des modèles d'apprentissage automatique s'appuie sur les données de capteurs de production (mesures dimensionnelles, températures, vitesses…). Cependant, dans le domaine industriel, ces capteurs génèrent forcément du « bruit » spatial et des valeurs aberrantes ; ce qui risque de fausser l'apprentissage des algorithmes de classification.
C'est dans cette optique que ce projet vise à proposer une analyse robuste qui met l'accent sur l'importance du traitement géométrique et du débruitage des données en amont pour garantir une prédiction fiable des pièces défectueuses.
Pour ce faire, nous proposons un pipeline séquentiel où un algorithme de détection de bruit par densités (DBSCAN) filtre les données d'entraînement avant qu'un classificateur à vecteurs de support (SVM) ne soit entraîné, ce qui permettra de quantifier l'impact du débruitage sur la qualité de la prédiction.

I. Objectifs

L'objectif principal est de réussir à évaluer et à quantifier l'impact du débruitage spatial sur les performances prédictives d'un modèle de classification, dans un contexte de données industrielles qui présentent un fort déséquilibre de classes.
On pourrait diviser ce projet en quatre objectifs spécifiques :

- **Explorer et visualiser** la structure des données en réduisant leur dimensionnalité par une analyse de composantes principales (PCA).
- **Détecter et retirer le bruit spatial** des données d'entraînement en utilisant l'algorithme DBSCAN.
- **Entraîner et comparer deux classificateurs SVM** (l'un sur les données brutes et l'autre sur les données débruitées) afin de mesurer l'impact du filtrage de données sur la qualité de la prédiction.
- **Évaluer les modèles** obtenus avec F1-score, AUC-ROC et la matrice de confusion.

Ces objectifs s'inscrivent dans une perspective d'application concrète des méthodes fondamentales en science des données, notamment en matière de prétraitement de données réelles, d'ingénierie de caractéristiques, ainsi que d'interprétation de visualisation des résultats.

II. Description des données analytiques

a. Source et structure de données

Nous utiliserons le jeu de données « AI4I 2020 Predictive Maintenance Dataset », un ensemble de données de haute qualité basé sur des paramètres industriels réels d'usinage. Ce jeu de données reflète les conditions de production rencontrées dans l'usinage de précision, où plusieurs variables physiques interagissent pour déterminer si la machine est défectueuse ou pas.

Caractéristiques des données

Caractéristique	Type de variables	Description
Type	Explicative	Niveau de qualité du produit (L : Low, M : Medium, H : High)
Température de l'air (K)	Explicative	Température de l'environnement de travail
Température du processus (K)	Explicative	Chaleur générée par l'interaction mécanique entre l'outil et la pièce
Vitesse de rotation (tr/min)	Explicative	Paramètre dynamique lié à la puissance moteur
Couple (Nm)	Explicative	Force de torsion appliquée
Usure de l'outil (min)	Explicative	Temps cumulé d'utilisation
Machine failure	Cible	État final de la pièce (0 : bon, 1 : mauvais)

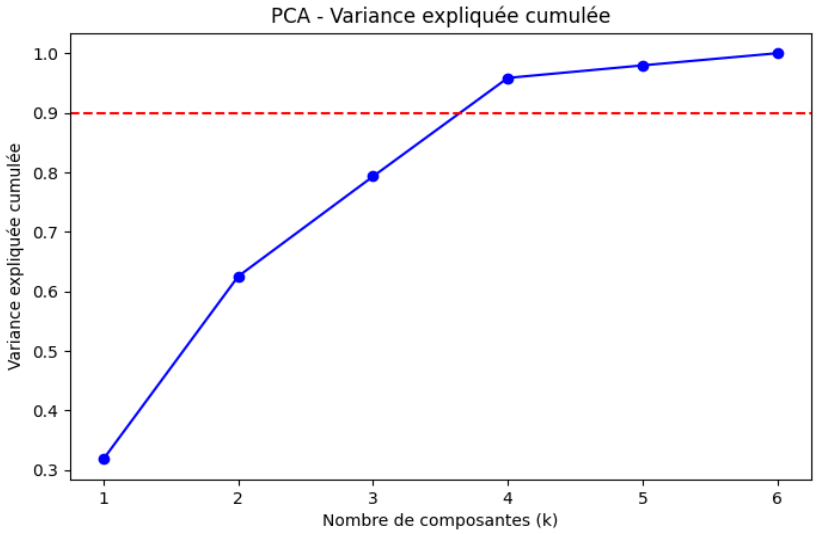
Tableau 1 - Caractéristiques des données

Résumé du jeu de données :

- Observations : 10 000 pièces usinées
- Défis : Le jeu de données présente deux défis algorithmiques majeurs. D'abord, un fort déséquilibre de classes (seulement 339 observations sur 10 000 correspondent à une défaillance, soit 3,39%). Et ensuite, la présence de variables fortement corrélées (par exemple, la température de l'air et la température du processus), ce qui justifie l'utilisation de la PCA.

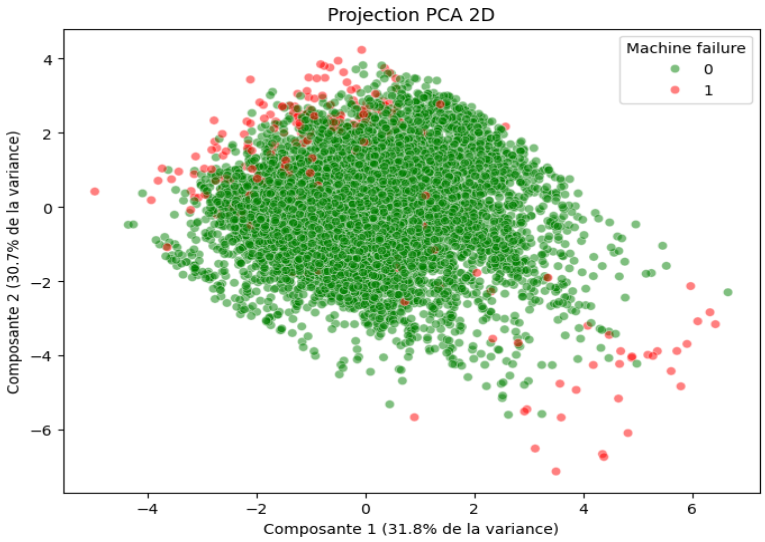
b. Exploration visuelle (PCA) et justification du débruitage

<u>Note</u> : Avant d'appliquer l'algorithme de PCA, les variables continues ont été standardisées afin de ramener leurs variances respectives à la même échelle.



1 - *PCA - Variance expliquée cumulée*

L'analyse de la variance expliquée cumulée démontre qu'il faut conserver au moins k = 4 composantes pour capturer 90 % de l'information dans le jeu de données. Mais pour faciliter la visualisation des données, une projection en 2D nous permet de retenir environ 62,5 % de la variance totale, ce qui est suffisant pour observer le comportement spatial des classes.



2 - Projection PCA 2D

La projection 2D met en évidence la complexité du problème de classification. L'on peut en tirer deux observations importantes grâce à ce graphique :

- Forte densité de la classe majoritaire : Les pièces conformes (points verts) forment un groupe très dense au centre du graphique

- Dispersion et bruit spatial de la classe minoritaire : Les pièces défectueuses (points rouges) ne sont pas facilement séparables et aussi ne forment pas un groupe distinct. Elles sont plutôt fortement dispersées et, pour la plupart, se situent à l'extrémité du groupe des points verts.

L'exploration des données nous confirme que de nombreuses défaillances se comportent comme du bruit. Un algorithme de classification classique entraîné directement sur ces données brutes aurait du mal à tracer une frontière de décision nette. Cela est assez pour justifier largement l'application d'un algorithme basé sur la densité, tel que DBSCAN, en prétraitement. Son rôle serait de, plus ou moins, identifier et filtrer les points de bruits isolés afin de « nettoyer » l'espace d'apprentissage avant l'entraînement du SVM.

III. Méthodologie

a. Prétraitement

Avant toute modélisation, les données ont nécessité deux transformations majeures :

- **Encodage ordinal :** La variable catégorique « Type » a été convertie en valeurs numériques par ordre croissant (L=0, M=1, H=2) afin de pouvoir quantifier cette variable et aussi préserver l'ordre logique de la qualité des produits.

- **Standardisation** : Les algorithmes basés sur la distance euclidienne (comme DBSCAN et SVM) sont très sensibles aux différentes échelles des variables dans le jeu de données. Ici, nous avons appliqué une standardisation ($Z = \frac{X-\mu}{\sigma}$) sur l'ensemble des variables explicatives pour nous assurer que les variables ayant de grandes valeurs ne dominent pas artificiellement le calcul des distances par rapport à des variables plus petites.

b. Division des données (Split train/test)

Avant d'appliquer les algorithmes de modélisation, nous avons décidé de diviser le jeu de données en deux sous-ensembles : un ensemble d'entraînement (80%, soit 8000 observations) et un ensemble de test (20%, soit 2000 observations). Cette division est effectuée tout en gardant le pourcentage de défaillances (3,39%) dans chaque groupe afin d'éviter que le déséquilibre des classes observé dans les données brutes disparaisse. L'algorithme de débruitage (DBSCAN) ne sera appliqué que sur l'ensemble d'entraînement. Cela permet d'éviter toute fuite de donnée, ce qui garantit que l'ensemble de test reste bien une simulation parfaite de nouvelles données brutes et non vues par le modèle.

c. Paramètres de DBSCAN

L'algorithme DBSCAN repose sur deux paramètres : le nombre minimum de points pour définir un groupe dense (minPts) et le rayon maximal du voisinage (ε).

Détermination de minPts

Pour les jeux de données multidimensionnels, il est recommandé de fixer ce paramètre à deux fois le nombre de dimensions de l'espace d'apprentissage [1]. Notre jeu de données contient 6 variables explicatives, alors on choisit minPts = 2 x 6 = 12.

Détermination de ε

Pour identifier le rayon optimal, nous avons appliqué la méthode de la « k-distance», qui consiste à calculer pour chaque observation des données d'entraînement la distance euclidienne la séparant de son 12e voisin le plus proche ; ensuite tracer ces distances en ordre croissant. « Cette méthode nous permet de repérer un point précis sur la courbe où le taux de variation des distances change significativement, indiquant la valeur epsilon appropriée ». [1]

```python
# Paramètres de DBSCAN
from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt
import numpy as np

min_pts = 12                                # min_pts = 2 * nombre de dimensions = 2 * 6 = 12

# Calcul des k plus proches voisins
nombres = NearestNeighbors(n_neighbors=min_pts)
nombres.fit(X_train)
distances, _ = nombres.kneighbors(X_train)

distances_k = distances[:, -1]                  # distance au k-ième plus proche voisin
distances_k_sorted = np.sort(distances_k)       # tri par odre croissant
plt.figure(figsize=(8, 3))
plt.plot(distances_k_sorted, color='steelblue')
plt.xlabel('Points triés')
plt.ylabel(f'Distance au {min_pts}-ième voisin')
plt.title('Graphique k-distance — Choix de epsilon')
plt.show()
```

Figure 3 - Paramètres de DBSCAN

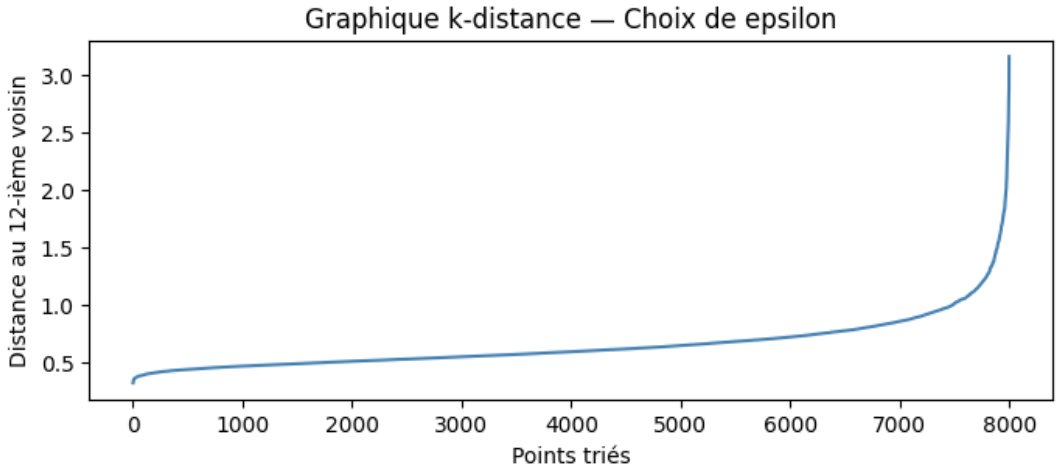


Figure 4 - Graphique k-distance - Choix d'epsilon

Le graphique obtenu présente une courbe presque horizontale qui représente la grande majorité des points situés dans l'amas dense (les points verts sur la figure 2 qui représente les pièces conformes). Ensuite, à l'extrémité droite, on observe une montée soudaine de la courbe, ce qui est une caractéristique des points isolés dans l'espace multidimensionnel.
Le point d'inflexion de cette courbe (le « coude »), qui marque la frontière mathématique entre la structure dense et le bruit spatial se situe visuellement entre les distances 1.0 et 1.3. Nous considérons la distance 1.0 pour la suite de notre étude. On retient donc les paramètres finaux : $\varepsilon = 1.0$ et minPts =12.

d. DBSCAN

```python
# PARTIE 5 : DBSCAN
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Paramètres choisis
epsilon = 1.0
min_pts = 12

dbscan = DBSCAN(eps=epsilon, min_samples=min_pts)

# Prédiction des étiquettes
labels = dbscan.fit_predict(X_train)         # application uniquement sur les données entraînées

# Filtre pour retirer le bruit (points étiquetés -1)
filtre = labels != -1                        # retourne TRUE si le point n'est pas du bruit

# Création des données filtrées
X_train_clean = X_train[filtre]
y_train_clean = y_train[filtre]
# Résultats
total_initial = len(X_train)
total_bruit = (labels == -1).sum()
pourcentage_bruit = (total_bruit / total_initial) * 100

print(f"Total de points initiaux : {total_initial}")
print(f"Total de points considérés comme bruit : {total_bruit}")
print(f"Pourcentage de bruit retiré : {pourcentage_bruit:.2f}%")
print(f"Taille finale de X_train_clean : {len(X_train_clean)} observations")
print(f"Défaillances avant nettoyage : {y_train.sum()}")
print(f"Défaillances après nettoyage : {y_train_clean.sum()}")
print(f"Défaillances retirées : {y_train.sum() - y_train_clean.sum()}")
print(f"Pourcentage de défaillances conservées : {(y_train_clean.sum() / y_train.sum()) * 100:.2f}%")
```

Figure 5 - Algorithme DBSCAN

L'algorithme DBSCAN a été appliquée seulement sur l'ensemble d'entraînement (8000 observations) afin d'éviter toute fuite d'information vers l'ensemble de test. L'objectif de cette étape est de retirer le bruit spatial pour faciliter la convergence du futur classificateur.
Avec les paramètres $\varepsilon = 1.0$ et minPts = 12, DBSCAN a identifié 202 points bruités sur 8 000 observations d'entraînement (soit 2,53%). Parmi les 271 défaillances initiales, 41 ont été retirées, ce qui signifie que 84,9% des défaillances sont conservées dans l'ensemble d'entraînement nettoyé.

e. SVM

Optimisation des hyperparamètres

```python
# PARTIE 6 : OPTIMISATION DES HYPERPARAMÈTRES POUR SVM
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

#Grilles d'hyperparamètres
grille = {
    'C': [0.1, 1, 10],                      # pénalité de l'erreur
    'gamma': [0.01, 0.1, 1],                # influence locale
    'kernel': ['rbf'],                      # noyau non linéaire. utilisation justifiée par PCA
    'class_weight': ['balanced']            # gestion du déséquilibre des classes
}

# SVM pour les deux modèles (brute et nettoyée)
svm_brute = SVC(probability=True, random_state=71)
svm_clean = SVC(probability=True, random_state=71)

# GridSearchCV pour les deux modèles
grid_search_brute = GridSearchCV(estimator=svm_brute,
                                 param_grid=grille,
                                 cv=5,                          # validation croisée à 5 plis
                                 scoring='f1',                  # optimisation du F1-score
                                 n_jobs=-1)

grid_search_clean = GridSearchCV(estimator=svm_clean,
                                 param_grid=grille,
                                 cv=5,
                                 scoring='f1',
                                 n_jobs=-1)

# Modèle 1 : Entraînement sur les données brutes (X_train, y_train)
grid_search_brute.fit(X_train, y_train)
meilleur_svm_brute = grid_search_brute.best_estimator_        # on garde le meilleur modèle trouvé

print(f"Meilleurs paramètres pour SVM 1 (données brutes) : {grid_search_brute.best_params_}")

# Modèle 2 : Entraînement sur les données nettoyées (X_train_clean, y_train_clean)
grid_search_clean.fit(X_train_clean, y_train_clean)
meilleur_svm_clean = grid_search_clean.best_estimator_       # on garde le meilleur modèle trouvé

print(f"Meilleurs paramètres pour SVM 2 (données nettoyées) : {grid_search_clean.best_params_}")
```

✓ 1m 25.0s

```
Meilleurs paramètres pour SVM 1 (données brutes) : {'C': 10, 'class_weight': 'balanced', 'gamma': 1, 'kernel': 'rbf'}
Meilleur score F1 en validation croisée pour SVM 1 (données brutes): 0.5119
Meilleurs paramètres pour SVM 2 (données nettoyées) : {'C': 10, 'class_weight': 'balanced', 'gamma': 1, 'kernel': 'rbf'}
Meilleur score F1 en validation croisée pour SVM 2 (données nettoyées): 0.5193
```

Figure 6 - Paramètres SVM

Afin de quantifier l'impact du débruitage, l'algorithme SVM sera entraîné d'une part sur les données standardisées (X_train), et d'autre part sur les données filtrées par DBSCAN (X_train_clean).
Pour garantir la validité de l'apprentissage, la configuration du modèle a été pensée pour mieux répondre aux défis de notre jeu de données :

- **Le choix du noyau (Kernel RBF) :** La PCA (voir figure 2) a clairement démontré que les classes ne sont pas séparables par un hyperplan linéaire. L'utilisation du noyau gaussien (RBF) est donc indispensable afin de nous

permettre de projeter mathématiquement les observations dans un espace de plus grande dimension pour y tracer des frontières de décision non linéaires. [3].

- **Le déséquilibre des classes :** Le défi majeur de l'industrie manufacturière est la rareté des pannes (environ 3.4% de défaillances dans notre échantillon).
  En vue de cela, un SVM classique chercherait à maximiser l'exactitude globale et pourrait classer toutes les pièces comme « conformes ». [4]
  Pour contrer ce problème, nous forçons une pénalisation asymétrique avec le paramètre « class_weight = 'balanced' ». Avec cela, le modèle accordera un poids mathématique beaucoup plus lourd à la classe minoritaire.

- **L'optimisation par recherche sur grille (GridSearchCV) :** La performance d'un SVM RBF dépend de deux paramètres : C (la pénalité de l'erreur) et γ (l'influence locale de chaque observation). Afin d'éviter tout biais humain ou surapprentissage, ces paramètres ne seront pas fixés arbitrairement. Ils seront déterminés par une recherche exhaustive sur une grille de valeurs, couplée à une validation croisée à 5 plis directement lors de l'entraînement. [6]

IV. Résultats

a. Résultats obtenus après DBSCAN

```
Total de points initiaux : 8000
Total de points considérés comme bruit : 202
Pourcentage de bruit retiré : 2.53%
Taille finale de X_train_clean : 7798 observations
Défaillances avant nettoyage : 271
Défaillances après nettoyage : 230
Défaillances retirées : 41
Pourcentage de défaillances conservées : 84.87%
```

Figure 7 - Résultats DBSCAN

L'algorithme a identifié 202 observations comme étant du bruit spatial, soit 2,53 % des données d'entraînement. X_train_clean, l'ensemble de données utilisé pour le second modèle contient donc 7798 observations au lieu de 8000 observations, puisque les 202 observations aberrantes ont été retirées.
Il est tout aussi important de noter l'impact de ce filtrage sur les défaillances. Avant le nettoyage, l'on avait 271 défaillances. Après le nettoyage, nous nous retrouvons avec 230 défaillances. Le retrait de ces 41 observations (environ 15% des cas critiques) confirme visuellement ce que suggérait la projection PCA (figure 2) : une proportion non négligeable des pannes se comporte géométriquement comme des anomalies

isolées. Par conséquent, le retrait de ces points devrait permettre au SVM de tracer une frontière de décision beaucoup plus généralisable.

b. Comparaison des deux modèles SVM

Les deux modèles SVM ont été évalués sur le même ensemble de test de 2000 observations, dont 68 défaillances réelles. Les résultats sont présentés dans le tableau suivant.

Métrique	SVM brut	SVM propre
F1-score	0.5465	0.5000
AUC-ROC	0.9304	0.9298

Tableau 2 - F1-score et AUC-ROC

Les deux modèles présentent une excellente aire sous la courbe (AUC-ROC ≈ 0.93). Cependant, la métrique F1-score indique une supériorité du modèle entraîné sur les données brutes.

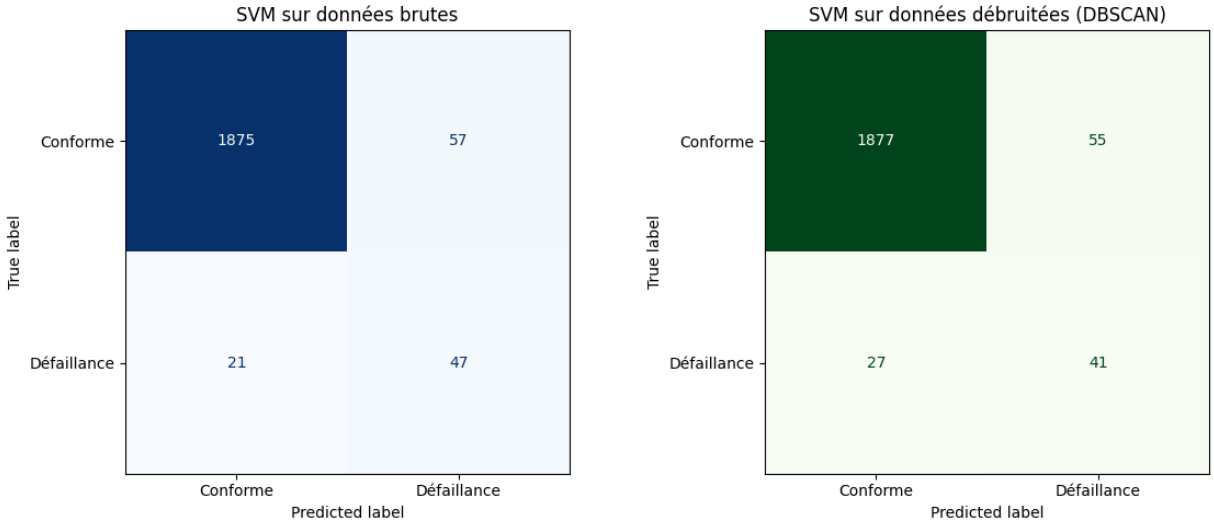


Figure 8 - Matrices de confusion

L'observation des matrices de confusion explique cette baisse de performance.
Le SVM brut détecte 47 défaillances sur 68 (environ 69.1% de rappel) et 57 faux positifs.
Le SVM propre n'en détecte que 41 défaillances sur 68 (environ 60.3% de rappel) et 55 faux positifs.
Dans un contexte industriel, un faux négatif (une défaillance non détectée) est beaucoup plus coûteux qu'un faux positif (une alarme plus ou moins inutile) ; ce qui renforce la supériorité du SVM brut comparé au SVM propre avec les données que nous avons.

c. Interprétation sur l'hypothèse de recherche

Contrairement à l'hypothèse initiale, le débruitage par DBSCAN n'a pas amélioré les performances du classificateur. Ce résultat peut être expliqué directement par la nature des données. En effet, comme la projection PCA l'avait montré (voir figure 2), les défaillances sont dispersées aux alentours du nuage dense. En isolant les points isolés comme « bruit », DBSCAN a retiré 41 sur 271 défaillances réelles, soit 15,1% des défaillances. Le SVM a donc appris avec moins d'exemples de la classe minoritaire, ce qui a baissé sa capacité à les détecter.

V. Conclusion

Ce projet a mis en œuvre un pipeline séquentiel combinant PCA, DBSCAN et SVM pour la détection de « machines non conformes » dans un contexte industriel déséquilibré.
Le résultat final obtenu est plutôt contre-intuitif, mais reste scientifiquement rigoureux : le débruitage par DBSCAN n'améliore pas, et dégrade même légèrement, les performances du SVM. En effet, le F1-score passe de 0.5465 pour les données brutes à 0.5000 pour les données débruitées.
Cette conclusion met en évidence une limite de DBSCAN appliqué à des anomalies dans le contexte industriel. On peut dire que les défaillances rares et dispersées autour de la zone dense sont géométriquement similaires au bruit spatial que DBSCAN cherche à éliminer. Par conséquent, le filtre de densité ne fait pas de différence entre les vrais points aberrants et les défaillances qui sont minoritaires dans le jeu de données.
Les résultats obtenus dans ce projet mettent en lumière un principe important en science des données : une méthode de prétraitement performante en théorie peut être contre-productive si elle entre en conflit avec la structure intrinsèque des données cibles.
Dans le contexte industriel, nous retenons qu'il reste préférable de conserver l'intégralité des données brutes et de laisser le SVM (avec noyau RBF) tracer ses propres frontières, plutôt que d'imposer un filtrage spatial strict en amont.

Une piste d'amélioration serait d'explorer des algorithmes de détection d'anomalies plus adaptés pour les jeux de données déséquilibrées, comme Isolation Forest, qui ne repose pas sur la densité spatiale et serait moins susceptible de confondre les défaillances minoritaires avec du bruit.

VI. Contributions

Ce projet a été réalisé de manière individuelle.

Bibliographie

1 Monko, G., & Kimura, M. (2023). *Optimized DBSCAN Parameter Selection : Stratified Sampling for Epsilon and GridSearch for Minimum Samples* (SSRN Scholarly Paper No. 4636699). Social Science Research Network. https://papers.ssrn.com/abstract=4636699

2 AI4I 2020 Predictive Maintenance Dataset [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C

3 Han, S., Qubo, C., & Meng, H. (2012). Parameter selection in SVM with RBF kernel function. *World Automation Congress 2012*, 1-4. https://ieeexplore.ieee.org/abstract/document/6321759

4 Madhavi, K. R., Ramrao, N., Kumar, T. K., Raju, K. S., & Sellathurai, M. (2025). *Proceedings of Sixth International Conference on Computer and Communication Technologies : IC3T 2024, Volume 2*. Springer Nature.

5 Ranjan, G. S. K., Kumar Verma, A., & Radhika, S. (2019). K-Nearest Neighbors and Grid Search CV Based Real Time Fault Monitoring System for Industries. *2019 IEEE 5th International Conference for Convergence in Technology (I2CT)*, 1-5. https://doi.org/10.1109/I2CT45611.2019.9033691

6 Azis, H. (2024). Assessing the Performance of Logistic Regression in Heart Disease Detection through 5-Fold Cross-Validation. *International Journal of Artificial Intelligence in Medical Issues*, *2*(1), 1-11. https://doi.org/10.56705/ijaimi.v2i1.137